

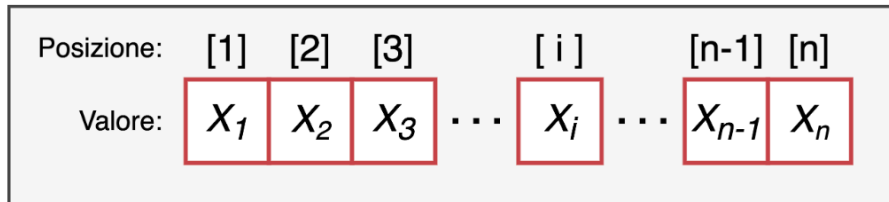
Modulo 1b – Introduzione a R

Ottavia M. Epifania
University of Trento
ottavia.epifania@unitn.it

A.A. 2026/2027

- 1 Vettori
- 2 Matrici
- 3 Dataframe
- 4 Liste
- 5 Importare dati esterni
- 6 Sintesi delle strutture tabellari

I vettori sono una struttura dati unidimensionale e sono la più semplice presente in R.

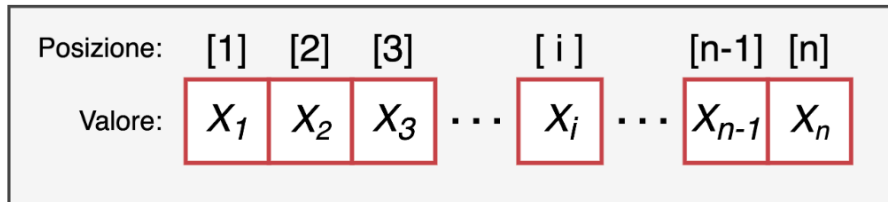


Caratteristiche:

- **la lunghezza:** il numero di elementi da cui è formato il vettore
- **la tipologia:** la tipologia di dati da cui è formato il vettore. Un vettore infatti deve essere formato da **elementi tutti dello stesso tipo!**

Caratteristiche degli elementi di un vettore

- **un valore**: il valore dell'elemento che può essere di qualsiasi tipo ad esempio un numero o una serie di caratteri
- **un indice di posizione**: un numero intero positivo che identifica la sua posizione all'interno del vettore.



Creare un vettore con c()

Concatena diversi oggetti insieme per creare un nuovo oggetto unico

```
x = c(1, 2, 3) ; x
```

```
[1] 1 2 3
```

```
X = 1:3 ; X
```

```
[1] 1 2 3
```

```
x == X
```

```
[1] TRUE TRUE TRUE
```

1 Vettori

- Tipologie di vettore
- Indicizzare i vettori

2 Matrici

3 Dataframe

4 Liste

5 Importare dati esterni

6 Sintesi delle strutture tabellari

Tipi di oggetti diversi → Tipi di vettori diversi:

- `num` (numeric): Numeri (-0.56, 3, 4.5)
- `logi` (logic): Logico (TRUE vs FALSE)
- `chr` (character): characters ("a", "b", "c")
- `factor` (factor): factor con diversi livelli

La distinzione più importanti è tra vettori di tipo categoriale (`chr` e `factor`) e vettori di tipo numerico (`int` e `num`):

- `integer` e `numeric`: Operazioni matematiche E calcolo delle frequenze
- `character` e `factor`: Calcolo delle frequenze

Tipologie di vettore

```
months = c(5, 6, 8, 10, 12, 16); months
```

```
[1]  5  6  8 10 12 16
```

```
weight = runif(6, 3, 18) ; weight
```

```
[1]  8.397962  6.677445 14.734444  4.343188 11.564218  3.048721
```

I vettori logici si ottengono testando delle condizioni sulla base dei valori di altri vettori:

```
months > 12
```

```
[1] FALSE FALSE FALSE FALSE FALSE  TRUE
```

Il vettore logico risultante può essere utilizzato

```
v_chr = c(letters[1:3], LETTERS[4:6]); v_chr
```

```
[1] "a" "b" "c" "D" "E" "F"
```

```
ses = factor(rep(c("low", "medium", "high"), each = 2)); ses
```

```
[1] low    low    medium medium high    high  
Levels: high low medium
```

Il secondo vettore è un vettore di tipo factor e prevede che si possa specificare l'ordine dei livelli

```
ses = factor(ses, levels = c("low", "medium", "high"))
```

Attenzione a non mischiare gli oggetti!!

`num + logi → num`

`num + factor → num`

`num + chr → chr`

`chr + logi → chr`

1 Vettori

- Tipologie di vettore
- Indicizzare i vettori

2 Matrici

3 Dataframe

4 Liste

5 Importare dati esterni

6 Sintesi delle strutture tabellari

```
nomi = c("Pasquale", "Egidio", "Debora", "Luca", "Andrea")  
nomi
```

```
[1] "Pasquale" "Egidio"    "Debora"    "Luca"      "Andrea"
```

Pasquale	Egidio	Debora	Luca	Andrea
----------	--------	--------	------	--------

1

2

3

4

5

```
nome_vettore[indice]
```

Pasquale	Egidio	Debora	Luca	Andrea
1	2	3	4	5

Indicizzare i vettori

Pasquale	Egidio	Debora	Luca	Andrea
1	2	3	4	5

`nomi[1]` → Pasquale

`nomi[3]` →

Pasquale	Egidio	Debora	Luca	Andrea
1	2	3	4	5

`nomi[1]` → Pasquale

`nomi[3]` → Debora

`nomi[seq(2, 5, by = 2)]` →

Pasquale	Egidio	Debora	Luca	Andrea
1	2	3	4	5

`nomi[1] → Pasquale`

`nomi[3] → Debora`

`nomi[seq(2, 5, by = 2)] → Egidio, Luca`

```
weight
```

```
[1]  8.397962  6.677445 14.734444  4.343188 11.564218  3.048721
```

```
weight[2] # secondo elemento di weight
```

```
[1] 6.677445
```

```
(weight[6] = 15.2) # sostituisce il sesto elemento di weight
```

```
[1] 15.2
```

```
weight[seq(1, 6, by = 2)] # elementi 1, 3, 5
```

```
[1]  8.397962 14.734444 11.564218
```

```
weight[2:6] # elementi dal 2 al 6 (incluso)
```

```
[1]  6.677445 14.734444  4.343188 11.564218 15.200000
```

Si può indicizzare con la logica

```
weight > 7
```

```
[1] TRUE FALSE TRUE FALSE TRUE TRUE
```

```
weight[weight > 7] # solo i valori > di 7
```

```
[1] 8.397962 14.734444 11.564218 15.200000
```

```
weight[weight >= 4.5 & weight < 8] # valori compresi tra 4.5 e 8
```

```
[1] 6.677445
```



In un nuovo documento quarto (`esercizi-modulo1.qmd`):

- Assegna i seguenti valori a un oggetto chiamato `my_var` (i valori devono essere **concatenati** tra loro)

23, 24, 25, 27, 28, 29, 30

- Calcola la media di `my_var` usando la funzione `mean()`, il minimo (`min()`) e massimo (`max()`)
- Crea un oggetto (`voto`) di tipo `character` come segue (usare la funzione `rep()`):
 - “si” ripetuto 12 volte
 - “no” ripetuto 15 volte

- 1 Vettori
- 2 Matrici**
- 3 Dataframe
- 4 Liste
- 5 Importare dati esterni
- 6 Sintesi delle strutture tabellari

Le matrici sono una struttura dati **bidimensionale** (caratterizzate da 2 dimensioni `dim()`) dove il numero di righe rappresenta la dimensione 1 e il numero di colonne la dimensione 2.

```
my_mat = matrix(data = 1:10, nrow = 2, ncol = 5)
my_mat
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	1	3	5	7	9
[2,]	2	4	6	8	10

```
`matrix(data, nrow, ncol, byrow = FALSE)`
```

```
# crea una matrice con 3 righe e 4 colonne  
# (i valori sono sistemati per riga)  
A = matrix(1:12, nrow=3, ncol = 4, byrow = TRUE)  
# assegna dei nomi alle righe  
rownames(A) = paste("a",  
                     1:nrow(A), sep = "_")  
# assegna dei nomi alle colonne  
colnames(A) = paste("b",  
                    1:ncol(A), sep = "_")
```

A

	b_1	b_2	b_3	b_4
a_1	1	2	3	4
a_2	5	6	7	8
a_3	9	10	11	12

1 Vettori

2 Matrici

- Caratteristiche
- Indicizzare le matrici

3 Dataframe

4 Liste

5 Importare dati esterni

6 Sintesi delle strutture tabellari

Caratteristiche

- Possono contenere una sola tipologia di dati
- Essendo bidimensionali, sono necessari due indici di posizione (righe e colonne) per identificare un elemento
- Possono essere viste come un insieme di singoli vettori

1 Vettori

2 Matrici

- Caratteristiche
- Indicizzare le matrici

3 Dataframe

4 Liste

5 Importare dati esterni

6 Sintesi delle strutture tabellari

Table 1: Struttura di una matrice

	[,1]	[,2]	[,3]
[1,]	[1, 1]	[1, 2]	[1, 3]
[2,]	[2, 1]	[2, 2]	[2, 3]
[3,]	[3, 1]	[3, 2]	[3, 3]

```
my_matrix[righe, colonne]
```

A

	b_1	b_2	b_3	b_4
a_1	1	2	3	4
a_2	5	6	7	8
a_3	9	10	11	12

 $A[1,] \rightarrow 1, 2, 3, 4$ $A[, 2] \rightarrow 2, 6, 10$ $A[2, 3] \rightarrow 7$



- Crea una matrice 3×3 con la tabellina del 3 fino a 24, assegnandola all'oggetto `my_mat`
- Dai un nome alle righe (e.g., `r1`, `r2`, `r3`) e alle colonne (e.g., `c1`, `c2`, `c3`)
- Indice da `my_mat`:
 - La prima riga
 - La seconda colonna
 - La cella `[1, 2]`

- 1 Vettori
- 2 Matrici
- 3 Dataframe**
- 4 Liste
- 5 Importare dati esterni
- 6 Sintesi delle strutture tabellari

Il dataframe è la struttura più “complessa”, utile e potente di R:

Come le matrici, è composto da due dimensioni (righe $\times$ colonne)

A differenza delle matrici, può contenere dati di tipo diverso

- Ogni elemento (colonna) è un vettore con un nome associato
- Ogni colonna deve avere lo stesso numero di elementi di conseguenza ogni riga ha lo stesso numero di elementi (struttura rettangolare)
- Tutte le colonne (quindi tutti i vettori contenuti nel dataframe) devono avere la stessa lunghezza

```
data.frame(. . .)
```

```
sexo = c(rep(c("M","F"), length=length(weight)))  
data = data.frame(id = paste0("id", 1:length(weight)),  
                  weight, sexo)  
data
```

	id	weight	sexo
1	id1	8.397962	M
2	id2	6.677445	F
3	id3	14.734444	M
4	id4	4.343188	F
5	id5	11.564218	M
6	id6	15.200000	F

- 1 Vettori
- 2 Matrici
- 3 Dataframe**
 - Indicizzare i dataframe
 - Attributi e caratteristiche
- 4 Liste
- 5 Importare dati esterni
- 6 Sintesi delle strutture tabellari

```
data[r, c], data$nome_variabile
```

```
data
```

	id	weight	sex
1	id1	8.397962	M
2	id2	6.677445	F
3	id3	14.734444	M
4	id4	4.343188	F
5	id5	11.564218	M
6	id6	15.200000	F

```
data[2, 1]
```

```
[1] "id2"
```

```
data$weight
```

```
[1] 8.397962 6.677445 14.734444 4
```

```
data$weight[2]
```

```
[1] 6.677445
```

La stessa logica con cui dai vettori si estraggono tutte le osservazioni che soddisfano un determinato requisito

```
weight[weight >= 7]
```

```
[1] 8.397962 14.734444 11.564218 15.200000
```

Nel caso del dataframe:

```
data[data$weight >=7, ]
```

	id	weight	sex
1	id1	8.397962	M
3	id3	14.734444	M
5	id5	11.564218	M
6	id6	15.200000	F

```
data[data$weight >=7 & data$weight < 9,  
      "id"]
```

```
[1] "id1"
```

- 1 Vettori
- 2 Matrici
- 3 Dataframe
 - Indicizzare i dataframe
 - Attributi e caratteristiche
- 4 Liste
- 5 Importare dati esterni
- 6 Sintesi delle strutture tabellari

Le funzioni `names()` , `dim()`, `nrow()`, `ncol()` sono alcune delle funzioni per ottenere informazioni sulle caratteristiche del dataframe.

La funzione più utile è `str()` poichè ci restituisce una veloce overview della struttura del dataframe: dimensioni, tipi di variabili,...

```
str(data)
```

```
'data.frame':   6 obs. of  3 variables:
 $ id      : chr  "id1" "id2" "id3" "id4" ...
 $ weight: num  8.4 6.68 14.73 4.34 11.56 ...
 $ sesso  : chr  "M" "F" "M" "F" ...
```

NB

La funzione `str()` funziona su tutti gli oggetti di R

- 1 Vettori
- 2 Matrici
- 3 Dataframe
- 4 Liste**
- 5 Importare dati esterni
- 6 Sintesi delle strutture tabellari

```
list(name1 = obj1, name2 = obj2, name3 = obj3, . . .)
```

Permette di creare liste di oggetti diversi (matrici, vettori, variabili) di diverso tipo (numeric, character, integer)

Molti dei risultati ottenuti con le funzioni per le analisi statistiche sono organizzati in liste, proprio perché permettono di contenere oggetti di natura diversa e con dimensionalità diversa

```
A <- c(1,8,5,0)
M <- matrix(A, 2, 2)
Nomi3 <- c("giorgio", "ugo", "anna")
LV <- c(TRUE, FALSE, TRUE, TRUE)
OgLista <- list(a = A, Mat = M, nomi = Nomi3, X = LV)
str(OgLista)
```

List of 4

```
$ a      : num [1:4] 1 8 5 0
$ Mat    : num [1:2, 1:2] 1 8 5 0
$ nomi   : chr [1:3] "giorgio" "ugo" "anna"
$ X      : logi [1:4] TRUE FALSE TRUE TRUE
```

- 1 Vettori
- 2 Matrici
- 3 Dataframe
- 4 Liste**
 - Indicizzare le liste
- 5 Importare dati esterni
- 6 Sintesi delle strutture tabellari

```
lista[[i]], lista$nome_oggettpo
```

[[i]]: indice dell'oggetto all'interno della lista a cui si vuole accedere

```
OgLista[[2]]
```

```
      [,1] [,2]  
[1,]    1    5  
[2,]    8    0
```

```
$nome_oggetto
```

```
OgLista$Mat
```

```
      [,1] [,2]  
[1,]    1    5  
[2,]    8    0
```

- 1 Vettori
- 2 Matrici
- 3 Dataframe
- 4 Liste
- 5 Importare dati esterni**
- 6 Sintesi delle strutture tabellari

- 1 Vettori
- 2 Matrici
- 3 Dataframe
- 4 Liste
- 5 Importare dati esterni**
 - Importa base
 - Importa alternativa 1
 - Importa alternativa 2
- 6 Sintesi delle strutture tabellari

```
read.csv2("file_path", header = T, sep = ";")
```

Il default della funzione prevede che la prima riga contenga i nomi delle variabili e che il separatore di colonne sia “;”.

Se si possiede un dataset chiamato “dati” nella cartella “data”, questo codice lo importerà all’interno dell’ambiente

```
# Lettura file dati esterni  
data <- read.csv2("data/dati.csv")
```

- 1 Vettori
- 2 Matrici
- 3 Dataframe
- 4 Liste
- 5 Importare dati esterni**
 - Importa base
 - Importa alternativa 1
 - Importa alternativa 2
- 6 Sintesi delle strutture tabellari

Vettori
oooooooooooooooo

Matrici
oooooooo

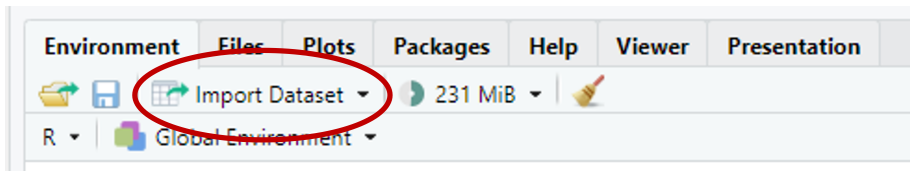
Dataframe
oooooooo

Liste
oooo

Importare dati esterni
oooo●oooo

Sintesi delle strutture tabellari
oooo

Importa alternativa 1



Vettori
oooooooooooooooo

Matrici
oooooooo

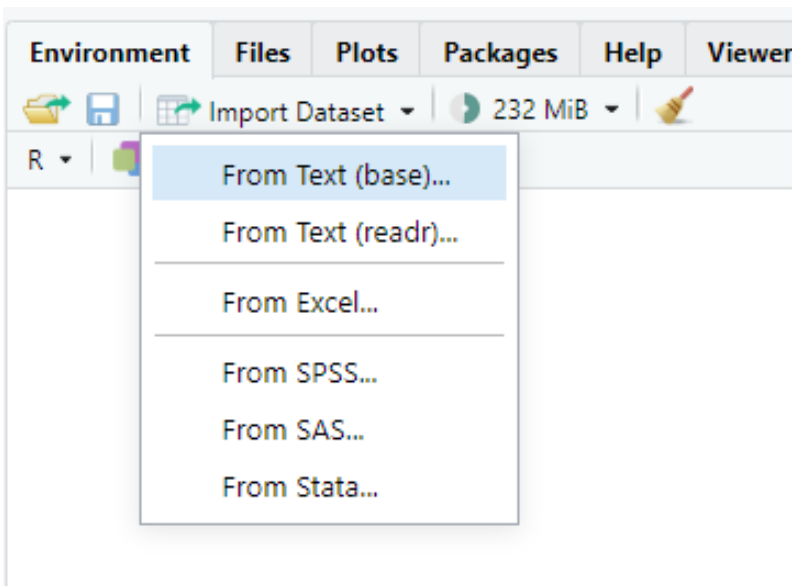
Dataframe
oooooooo

Liste
oooo

Importare dati esterni
oooo●oooo

Sintesi delle strutture tabellari
oooo

Importa alternativa 1



Vettori
○○○○○○○○○○○○○○○○○○

Matrici
○○○○○○○○○○

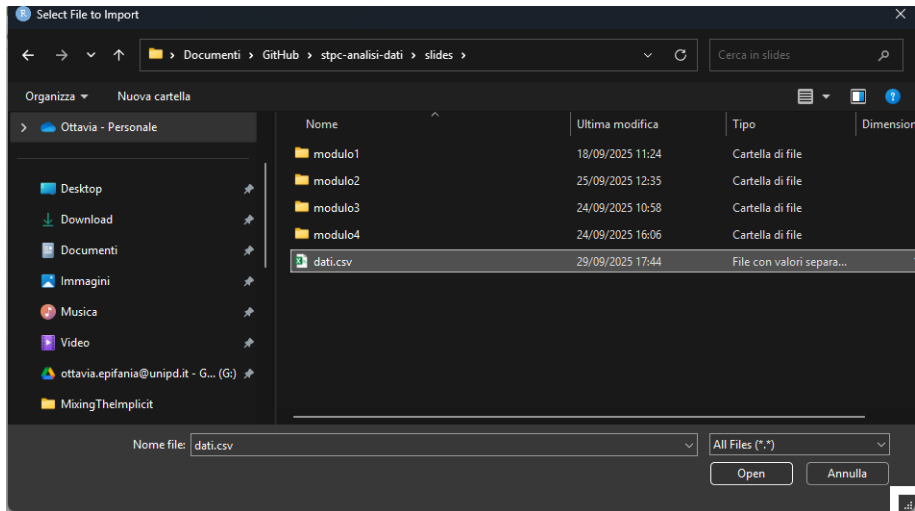
Dataframe
○○○○○○○○

Liste
○○○○○

Importare dati esterni
○○○○○○●○○○

Sintesi delle strutture tabellari
○○○○○○

Importa alternativa 1



Importa alternativa 1

Import Dataset

Name

Encoding

Heading ☒ Yes ☐ No

Row names

Separator

Decimal

Quote

Comment

na.strings

☐ Strings as factors

Input File

```
ID;eta;sex;regione  
sb1;18;M;nord  
sb2;19;F;nord  
sb3;21;F;nord  
sb4;18;F;nord  
sb5;19;M;centro  
sb6;22;M;sud  
sb7;18;M;centro  
sb8;20;F;sud  
sb9;18;F;nord  
sb10;19;F;centro  
sb11;20;M;nord  
sb12;18;M;sud  
sb13;18;M;sud  
sb14;19;F;centro  
sb15;21;F;centro
```

Data Frame

ID	eta	sex	regione
sb1	18	M	nord
sb2	19	F	nord
sb3	21	F	nord
sb4	18	F	nord
sb5	19	M	centro
sb6	22	M	sud
sb7	18	M	centro
sb8	20	F	sud
sb9	18	F	nord
sb10	19	F	centro
sb11	20	M	nord
sb12	18	M	sud
sb13	18	M	sud
sb14	19	F	centro
sb15	21	F	centro

Import Cancel

- 1 Vettori
- 2 Matrici
- 3 Dataframe
- 4 Liste
- 5 Importare dati esterni**
 - Importa base
 - Importa alternativa 1
 - Importa alternativa 2
- 6 Sintesi delle strutture tabellari

```
dati = read.csv("C:/Users/Ottavia/Documents/GitHub/stpc-analisi-c  
              sep=";", dec=",", header = TRUE)  
head(dati)
```

	ID	eta	sex	regione
1	sb1	18	M	nord
2	sb2	19	F	nord
3	sb3	21	F	nord
4	sb4	18	F	nord
5	sb5	19	M	centro
6	sb6	22	M	sud

- 1 Vettori
- 2 Matrici
- 3 Dataframe
- 4 Liste
- 5 Importare dati esterni
- 6 Sintesi delle strutture tabellari**

summary()

```
summary(data)
```

Riassume il contenuto di strutture di dati

```
summary(rock)
```

area	peri	shape	perm
Min. : 1016	Min. : 308.6	Min. :0.09033	Min. : 6.30
1st Qu.: 5305	1st Qu.:1414.9	1st Qu.:0.16226	1st Qu.: 76.45
Median : 7487	Median :2536.2	Median :0.19886	Median : 130.50
Mean : 7188	Mean :2682.2	Mean :0.21811	Mean : 415.45
3rd Qu.: 8870	3rd Qu.:3989.5	3rd Qu.:0.26267	3rd Qu.: 777.50
Max. :12212	Max. :4864.2	Max. :0.46413	Max. :1300.00

str()

```
str(data)
```

Fornisce informazioni circa la struttura generale di un oggetto:

```
str(rock)
```

```
'data.frame':   48 obs. of  4 variables:
 $ area : int   4990 7002 7558 7352 7943 7979 9333 8209 8393 6425
 $ peri : num   2792 3893 3931 3869 3949 ...
 $ shape: num   0.0903 0.1486 0.1833 0.1171 0.1224 ...
 $ perm : num    6.3 6.3 6.3 6.3 17.1 17.1 17.1 17.1 119 119 ...
```

head() (α) and tail() (ω)

head()

Le prime 6 osservazioni del dataset

```
head(rock)
```

	area	peri	shape	perm
1	4990	2791.90	0.0903296	6.3
2	7002	3892.60	0.1486220	6.3
3	7558	3930.66	0.1833120	6.3
4	7352	3869.32	0.1170630	6.3
5	7943	3948.54	0.1224170	17.1
6	7979	4010.15	0.1670450	17.1

tail()

le ultime 6 osservazioni del dataset

```
tail(rock)
```

	area	peri	shape	perm
43	5605	1145.690	0.464125	1300
44	8793	2280.490	0.420477	1300
45	3475	1174.110	0.200744	580
46	1651	597.808	0.262651	580
47	5514	1455.880	0.182453	580
48	9718	1485.580	0.200447	580

apply()

```
apply(data, MARGIN, FUN)
```

data: i dati su cui si vuole lavorare

MARGIN: 1 applica la funzione sulle righe (attraverso le colonne), 2 applica la funzione sulle colonne (attraverso le righe)

FUN: la funzione che si vuole applicare

```
apply(rock, 1, sum)    # Somma per riga
```

```
[1] 7788.290 10901.049 11495.143 11227.737 11908.762 12006.417 13696.040
[8] 12571.014 12194.244 9642.812 13963.201 12729.388 14770.169 12468.672
[15] 13498.943 12585.623 15629.464 15589.483 16801.020 14405.155 11408.854
[22] 16371.431 17051.890 11893.602 9077.671 6312.529 10072.432 7571.553
[29] 9250.435 7025.001 9393.452 10245.336 8356.958 7258.715 9692.914
[36] 10305.336 4945.877 2044.761 4813.624 7012.214 7289.872 2624.872
[43] 8051.154 12373.910 5229.311 2829.071 7550.062 11783.780
```

```
apply(rock, 2, mean)  # Media per colonna
```

```
          area          peri          shape          perm
7187.7291667 2682.2119375    0.2181104    415.4500000
```